Quiz

- Question #1: What is the value of the following expression?

  +(1,1)

- Wrong answer: **0**
- Wrong answer: **42**
- Answer: **2**

Quiz

- Question #2: What is the value of the following expression?

  + **proc** 8

- Wrong answer: **error**

- Answer: Trick question! + **proc** 8 is not an expression

Language Grammar for Quiz

<expr> ::= <num>
::= <bool>
::= <id>
::= <prim> ({ <expr> }*(,))
::= **proc** (<id>*(,)) <expr>
::= (<expr> <expr>*)
::= **if** <expr> **then** <expr> **else** <expr>
<prim> ::= + | - | * | **add1** | **sub1**
<bool> ::= **true** | **false**

Quiz

- Question #3: Is the following an expression?

add1(1, 7)

- Wrong answer: **No**

- Answer: **Yes** (according to our grammar)

Quiz

- Question #4: What is the value of the following expression?

  add1(1, 7)

- Answer: **2** (according to our interpreter)

- But no *real* language would accept **add1**(1, 7)

- Let's agree to call **add1**(1, 7) an ***ill-formed expression*** because **add1** should be used with only one argument

- Let's agree to never evaluate ill-formed expressions

Quiz

- Question #5: What is the value of the following expression?

add1(1, 7)

- Answer: **None** - the expression is ill-formed

Quiz

- Question #6: Is the following a well-formed expression?

+(**proc**(**x**)**x**, 5)

- Answer: **Yes**

Quiz

- Question #7: What is the value of the following expression?

  +(**proc**(**x**)**x**, 5)

- Answer: **None** - it produces an error:

  +: expects type <number> as 1st argument,
  given: (closure ((cbv-var x)) (var-exp x)
  (empty-env-record)); other arguments were: 5

- Let's agree that a **proc** expression cannot be inside a + form

Quiz

- Question #8: Is the following a well-formed expression?

+(**proc**(**x**)**x**, 5)

- Answer: **No**

Quiz

- Question #9: Is the following a well-formed expression?

  +((**proc**(**x**)**x** 7), 5)

- Answer: Depends on what we meant by *inside* in our most recent agreement
  - *Anywhere inside* - **No**
  - *Immediately inside* - **Yes**

- Since our intrepreter produces **12**, and since that result makes sense, let's agree on *immediately inside*

Quiz

- Question #10: Is the following a well-formed expression?

+((**proc**(**x**)**x true**), 5)

- Answer: **Yes**, but we don't want it to be!

Quiz

- Question #11: Is it possible to define ***well-formed*** (as a decidable property) so that we reject all expressions that produce errors?

- Answer: **Yes**: reject *all* expressions!

Quiz

- Question #12: Is it possible to define ***well-formed*** (as a decidable property) so that we reject *only* expressions that produce errors?

- Answer: **No**

+(1, **if ... then** 1 **else proc**(**x**)**x**)

- If we always knew whether **...** produces true or false, we could solve the halting problem

Types

- Solution to our dilemma
  - In the process of rejecting expressions that are certainly bad, also reject some expressions that are good

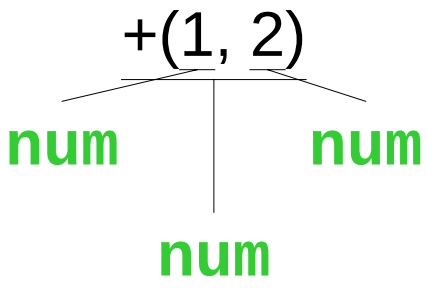
    +(1, **if** (**prime?** 131101) **then** 1 **else** **proc**(**x**)**x**)

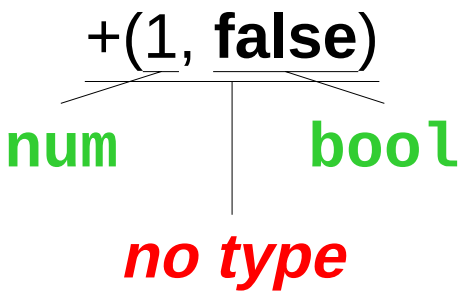
- Overall strategy:
  - Assign a ***type*** to each expression *without evaluating*
  - Compute the type of a complex expression based on the types of its subexpressions

Types

1 : num

true : bool





Type Rules

<num> : **num**

<bool> : **bool**

$$\frac{\langle expr\rangle_1 : \textbf{num} \qquad \langle expr\rangle_2 : \textbf{num}}{+(\langle expr\rangle_1, \langle expr\rangle_2) : \textbf{num}}$$

1 : **num**

true : **bool**

$$\frac{1 : \textbf{num} \qquad 2 : \textbf{num}}{+(1, 2) : \textbf{num}}$$

$$\frac{1 : \textbf{num} \qquad \textbf{false} : \textbf{bool}}{+(1, \textbf{false}) : \textit{no type}}$$

Type Rules

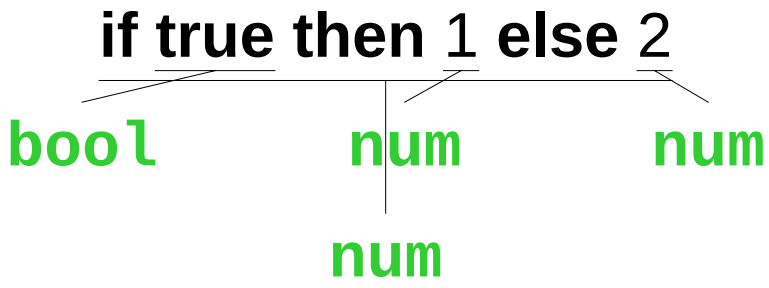
<num> : **num**

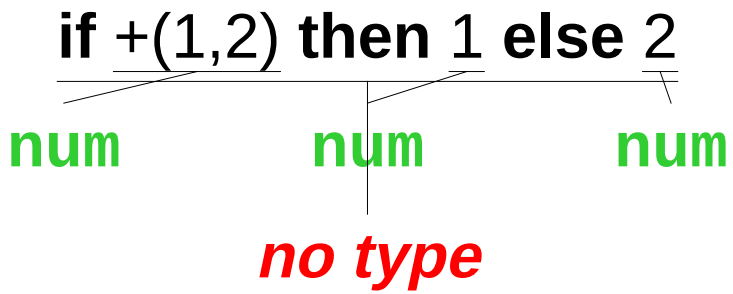
<bool> : **bool**

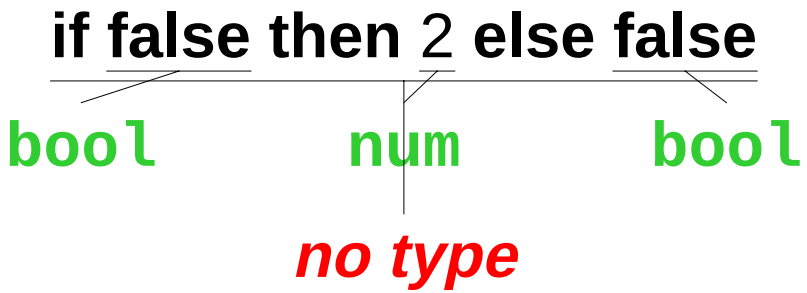
$$\frac{\langle expr\rangle_1 : \mathbf{num} \qquad \langle expr\rangle_2 : \mathbf{num}}{+(\langle expr\rangle_1, \langle expr\rangle_2) : \mathbf{num}}$$

$$\frac{\dfrac{1 : \mathbf{num} \qquad 2 : \mathbf{num}}{+(1, 2) : \mathbf{num}} \qquad 3 : \mathbf{num}}{+(+(1, 2),3) : \mathbf{num}}$$

Types: Conditionals







Conditional Type Rules

$$\frac{\langle expr\rangle_1 : \texttt{bool} \qquad \langle expr\rangle_2 : \langle type\rangle_0 \qquad \langle expr\rangle_3 : \langle type\rangle_0}{\textbf{if } \langle expr\rangle_1 \textbf{ then } \langle expr\rangle_2 \textbf{ else } \langle expr\rangle_3 : \langle type\rangle_0}$$

$$\frac{\textbf{true} : \texttt{bool} \qquad 1 : \texttt{num} \qquad 2 : \texttt{num}}{\textbf{if true then } 1 \textbf{ else } 2 : \texttt{num}}$$

$$\frac{+(1,2) : \texttt{num} \qquad 1 : \texttt{num} \qquad 2 : \texttt{num}}{\textbf{if } +(1,2) \textbf{ then } 1 \textbf{ else } 2 : \textit{no type}}$$

$$\frac{\textbf{false} : \texttt{bool} \qquad 2 : \texttt{num} \qquad \textbf{false} : \texttt{bool}}{\textbf{if false then } 2 \textbf{ else false} : \textit{no type}}$$

Types: Variables and Functions

x : ***no type***

proc(bool x)x

`bool`

`(bool → bool)`

proc(bool x)if x then 1 **else** 2

`bool` `num` `num`

`num`

`(bool → num)`

Variable and Function Type Rules

$$\{ \ldots \langle \text{id} \rangle : T \ldots \} \vdash \langle \text{id} \rangle : T$$

$$\frac{\{ \langle \text{id} \rangle : T_1 \} + E \vdash e : T_2}{E \vdash \textbf{proc}(T_1 \langle \text{id} \rangle) e : (T_1 \rightarrow T_2)}$$

Abbreviations: T = <type> e = <expr> E = <env>

Variable and Function Type Rules

$$\{ \ldots \langle id \rangle : T \ldots \} \vdash \langle id \rangle : T$$

$$\frac{\{ \langle id \rangle : T_1 \} + E \vdash e : T_2}{E \vdash \textbf{proc}(T_1 \langle id \rangle) e : (T_1 \rightarrow T_2)}$$

$$\{ \} \vdash \textbf{x} : \textit{no type}$$

$$\frac{\{\textbf{x} : \texttt{bool}\} \vdash \textbf{x} : \texttt{bool}}{\{ \} \vdash \textbf{proc(bool x)x} : (\texttt{bool} \rightarrow \texttt{bool})}$$

$$\frac{\dfrac{\{\textbf{x} : \texttt{bool}\} \vdash \textbf{x} : \texttt{bool} \quad \{\textbf{x} : \texttt{bool}\} \vdash 1 : \texttt{num} \quad \{\textbf{x} : \texttt{bool}\} \vdash 2 : \texttt{num}}{\{\textbf{x} : \texttt{bool}\} \vdash \textbf{if x then } 1 \textbf{ else } 2 : \texttt{num}}}{\{ \} \vdash \textbf{proc(bool x)if x then } 1 \textbf{ else } 2 : (\texttt{bool} \rightarrow \texttt{num})}$$

Resvised Rules

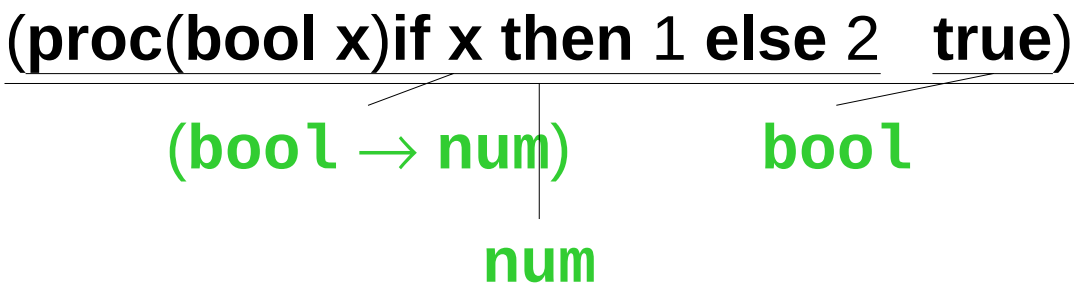
$$E \vdash \text{<num>} : \texttt{num}$$

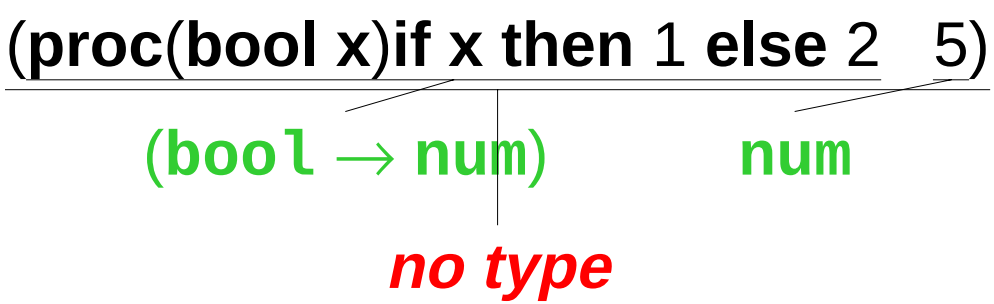
$$E \vdash \text{<bool>} : \texttt{bool}$$

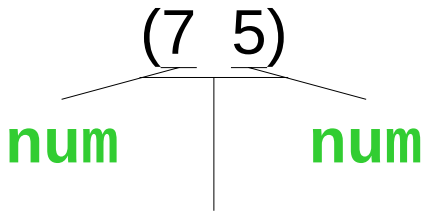
$$\frac{E \vdash e_1 : \texttt{num} \qquad E \vdash e_1 : \texttt{num}}{E \vdash +(e_1, e_2) : \texttt{num}}$$

$$\frac{E \vdash e_1 : \texttt{bool} \qquad E \vdash e_2 : T_0 \qquad E \vdash e_3 : T_0}{E \vdash \textbf{if } e_1 \textbf{ then } e_2 \textbf{ else } e_3 : T_0}$$

Types: Function Calls







Function Call Type Rule

$$\frac{E \vdash e_1 : (T_2 \rightarrow T_3) \qquad E \vdash e_2 : T_2}{E \vdash (e_1\ e_2) : T_3}$$

$$\frac{\{\} \vdash \textbf{proc(bool x)if x then } 1 \textbf{ else } 2 : (\texttt{bool} \rightarrow \texttt{num}) \qquad \{\} \vdash \textbf{true} : \texttt{bool}}{\{\} \vdash (\textbf{proc(bool x)if x then } 1 \textbf{ else } 2 \quad \textbf{true}) : \texttt{num}}$$

$$\frac{\{\} \vdash \textbf{proc(bool x)if x then } 1 \textbf{ else } 2 : (\texttt{bool} \rightarrow \texttt{num}) \qquad \{\} \vdash 5 : \texttt{num}}{\{\} \vdash (\textbf{proc(bool x)if x then } 1 \textbf{ else } 2 \quad 5) : \textit{no type}}$$

$$\frac{\{\} \vdash 7 : \texttt{num} \qquad \{\} \vdash 5 : \texttt{num}}{\{\} \vdash (7 \quad 5) : \textit{no type}}$$

Types: Multiple Arguments

proc(num x, num y)+(x, y)

num num

num

$(\text{num} \times \text{num} \to \text{num})$

(proc(num x, num y)+(x, y) 5 6)

$(\text{num} \times \text{num} \to \text{num})$ num num

num

(proc(num x, num y)+(x, y) 5)

$(\text{num} \times \text{num} \to \text{num})$ num

no type

Revised Function and Call Rules

$$\frac{\{ \langle id \rangle_1 : T_1, \ldots \langle id \rangle_n : T_n \} + E \vdash e : T_0}{E \vdash \textbf{proc}(T_1 \langle id \rangle_1, \ldots T_n \langle id \rangle_n) e : (T_1 \times \ldots T_n \rightarrow T_0)}$$

$$\frac{E \vdash e_0 : (T_1 \times \ldots T_n \rightarrow T_0) \qquad E \vdash e_1 : T_1 \qquad \ldots \qquad E \vdash e_n : T_n}{E \vdash (e_0\ e_1\ \ldots\ e_n) : T_0}$$

New Interpreter and Checker

- Change our interpreter:
  - Add types for arguments and letrec results to the grammar
- Implement a type-checker:
  - Produces the same type that the rules procedure
  - Calls itself recusrively to get types for sub-expressions
  - Treat primitives as built-in functions

$$+ : (\texttt{num} \times \texttt{num} \rightarrow \texttt{num})$$